

AI Toy Companion

Technical Update | Sprint Review and Architecture Report

Date	10 June 2026
Project	AI Toy Companion Android App with ESP32-S3 WiFi Voice Hardware
Prepared by	Machadev
Status	Stable WiFi build complete and released. Voice pipeline operational.
Delivery	Release APK ready for device installation and testing

Table of Contents

- 1. Executive Summary
- 2. Your Questions | Direct Answers
- 3. How the Voice Pipeline Works
- 4. Transport Layer | BLE to WiFi Migration
- 5. Gemini Live API Integration
- 6. Opus Encoding and Audio Delivery
- 7. Streaming Experiment | What We Attempted and Why It Did Not Move Forward
- 8. Root Cause Summary | Why Streaming Was Not Stable
- 9. Stable Architecture | What Is Running Now
- 10. Sprint Deliverables
- 11. Response Latency | Current Profile and Honest Assessment
- 12. Next Steps

1. Executive Summary

This document is a direct technical response to the questions raised after our previous architecture overview. It clarifies how the Gemini Live API is being used, corrects a misleading latency breakdown table from an earlier report, and gives an honest account of an experimental streaming approach that was designed, implemented, and ultimately not taken forward due to hardware stability constraints on the ESP32.

The sprint successfully delivered a complete WiFi TCP transport layer replacing BLE, automatic device discovery, a stable Opus audio encode and playback pipeline, and a release APK ready for on device testing. The pipeline consistently delivers end to end voice responses in 7 to 8 seconds. This is the production baseline.

SPRINT STATUS

Sprint outcome

WiFi TCP transport: delivered and stable.

Automatic device discovery: delivered.

Native Gemini audio to audio voice pipeline: confirmed and running.

Release APK: built and ready for install.

End to end latency: 7 to 8 seconds. This is the stable production baseline.

2. Your Questions | Direct Answers

Q1 | Is the pipeline native audio to audio, or a cascaded STT then LLM then TTS chain?

ANSWER

Native audio to audio via Gemini Multimodal Live API.

Model: gemini-2.0-flash-live-001 via BidiGenerateContent WebSocket.

Audio enters at 16 kHz. Audio response returns at 24 kHz.

There is no speech to text step, no separate LLM text call, no text to speech step.

One WebSocket. Voice in. Voice out.

The STT / Gemini text / TTS cost breakdown in the previous report described a candidate cascaded architecture we evaluated before choosing the Live API route.

It should not have appeared in the final document. We apologise for the confusion.

Q2 | Is audio played as it streams in, or buffered until the full response arrives?

ANSWER

Currently: the full Opus response is received before playback begins.

We designed and built a streaming ring buffer approach to change this.

After on device testing it produced unstable behaviour on the ESP32 hardware.

We made the decision not to take it forward and reverted to the proven approach.

Section 7 covers exactly what was attempted and why it was set aside.

Q3 | Is there a relay server adding delay?

ANSWER

The Gemini Live API is called directly from the phone. No relay.

The phone opens a WebSocket connection straight to Google's servers.

There is no intermediary server that the audio passes through.

The Supabase Edge Function handles Opus compression only. It receives the completed AI audio after Gemini finishes, compresses it, and returns it.

It does not sit between the app and Gemini. It does not add to the Gemini wait time.

Q4 | Where does the time go in a 7 to 8 second response?

Stage	Approx Time	Notes
User audio recorded and sent to app	~200 ms	16 kHz PCM over local WiFi TCP
Gemini Live API first audio token	500 to 900 ms	Model processing time. Inherent to the API.
Full AI response audio generation	2 000 to 4 000 ms	Depends on response length
Opus encode	150 to 300 ms	Runs in parallel. Not in the critical path.
TCP transfer to ESP32	100 to 200 ms	Local WiFi. Fast.
Playback on speaker	2 000 to 4 000 ms	Real time audio. Matches response length.
Total perceived delay	7 000 to 8 000 ms	From end of user speech to first word heard

The dominant factor is that playback cannot begin until the full audio file has been received and decoded. A 4 second AI response requires the full 4 seconds of audio to arrive before the speaker starts. That is the architectural characteristic of the current stable build.

3. How the Voice Pipeline Works

The complete pipeline is orchestrated by the mobile app. The ESP32 captures voice, the app manages the AI conversation, and the encoded audio response is delivered back to the device over WiFi.

Step	Component	What Happens
1. Record	INMP441 microphone I2S	User speaks. 16 kHz 16 bit PCM captured.
2. Send	ESP32 WiFi TCP	PCM audio streamed to the phone app over local WiFi.
3. AI call	Gemini Live API WebSocket	Audio sent. AI generates audio response. Native audio to audio.
4. Encode	Supabase Edge Function	24 kHz PCM response encoded to Opus for compact transfer.
5. Deliver	App to ESP32 WiFi TCP	Opus packets sent to firmware in length prefixed format.
6. Play	MAX98357A speaker I2S	Firmware decodes Opus and plays audio on the speaker.

Opus wire format

All audio sent from the app to the ESP32 uses a simple framing format. Each Opus packet is preceded by its 2 byte length, allowing the firmware to reliably reassemble packets regardless of how TCP splits the byte stream.

Opus TCP wire format

Each packet on the wire:

```
[ 2 bytes: packet length, little endian ][ N bytes: Opus data ]
```

Example | 87 byte Opus packet:

```
0x57 0x00 (87 in little endian)
```

```
...87 bytes of Opus compressed audio...
```

Repeated for each frame. The firmware reads the length first, then reads exactly that many bytes before decoding.

4. Transport Layer | BLE to WiFi Migration

The previous version of the toy used Bluetooth Low Energy for audio transfer. BLE had hard limits that made it unsuitable for streaming audio at 16 kHz. The migration to WiFi TCP removed all of these constraints.

Metric	BLE (previous)	WiFi TCP (current)
Maximum chunk size	500 bytes	~1 460 bytes (TCP segment)
Transfer speed for 240 KB audio	~4.8 seconds	~200 ms
Delay between writes	10 ms forced delay	None required
Connection reliability	Packet loss under interference	TCP guaranteed delivery
Device discovery	BLE scan (slow, manual pairing)	Automatic: UDP broadcast then mDNS then fallback

Automatic device discovery | No manual IP setup

One of the usability improvements in this sprint was eliminating the need to manually enter the ESP32 IP address. The app now discovers the device automatically using three methods tried in sequence.

```
Esp32DiscoveryService.ts | discoverHost()
```

```
// Esp32DiscoveryService.ts | Three tier automatic discovery

// Tier 1: ESP32 broadcasts its own IP on UDP port 8766
//   Firmware sends: 'ESP32AUDIO:192.168.1.44:8765'
//   App listens for 3.5 seconds
const udpResult = await this.tryUdpBroadcast(8766, 3500);
if (udpResult) return udpResult;

// Tier 2: Android NSD (mDNS) _esp32audio._tcp service
const nsdResult = await this.tryNsdDiscovery('_esp32audio._tcp.', 3500);
if (nsdResult) return nsdResult;

// Tier 3: Try known fallback IP addresses directly
for (const ip of ESP32_FALLBACK_IPS) {
  const ok = await Wi-FiTCPService.connect(ip, 8765);
  if (ok) return { host: ip, port: 8765, type: 'fallback' };
}
```

5. Gemini Live API Integration

The GeminiLiveService manages the persistent WebSocket connection to Google's Multimodal Live API. A single WebSocket session is kept open across multiple voice turns, which avoids the connection setup cost on every interaction.

Session kept warm between turns

Each time the user speaks, the service checks whether a WebSocket session is already open and reuses it rather than creating a new one. This was one of the improvements delivered this sprint.

```
GeminiLiveService.ts | Warm session
```

```
// GeminiLiveService.ts | Session reuse
async connect(systemPrompt: string): Promise<void> {
  if (this.ws && this.ready) {
    // Session already open | reuse it, no reconnect needed
    return;
  }
  // Open new WebSocket and wait for setupComplete event
  // This cost is now paid once at app startup, not on every turn
  const url = `${GEMINI_LIVE_WS_BASE}?key=${apiKey}`;
  this.ws = new WebSocket(url);
  // ... session setup ...
}
```

Audio in | Audio out

The user's voice is streamed to Gemini as raw PCM chunks. The API returns audio response chunks incrementally. The service collects those chunks and assembles the complete response for encoding once the turn is complete.

GeminiLiveService.ts | Audio streaming

```
// GeminiLiveService.ts | Send user audio to Gemini
streamPcm(pcmBuffer: ArrayBuffer): void {
  const b64 = btoa(String.fromCharCode(...new Uint8Array(pcmBuffer)));
  this.ws!.send(JSON.stringify({
    realtimeInput: {
      mediaChunks: [{ mimeType: 'audio/pcm;rate=16000', data: b64 }]
    }
  }));
}

// Gemini responds with audio chunks in handleMessage
// Chunks are assembled until turnComplete is received
// Complete 24 kHz PCM response then goes to the Opus encoder
```

6. Opus Encoding and Audio Delivery

Gemini returns audio at 24 kHz PCM. The Supabase Edge Function encodes this to Opus format for efficient transfer to the ESP32. Opus achieves roughly 10x compression compared to raw PCM, reducing a 5 second response from approximately 480 KB to around 15 KB, which transfers in under 200 ms on local WiFi.

encode-opus/index.ts | Supabase Edge Function

```
// encode-opus/index.ts | Supabase Edge Function
// Accepts: raw 16 kHz 16 bit mono PCM as POST body
// Returns: length prefixed Opus packet stream

const SAMPLE_RATE = 16000;
const FRAME_SAMPLES = 320; // 20 ms per frame at 16 kHz
const FRAME_BYTES = FRAME_SAMPLES * 2; // 640 bytes
const BITRATE = 24000; // 24 kbps

const encoder = new OpusScript(SAMPLE_RATE, 1, OpusScript.Application.VOIP);
encoder.setBitrate(BITRATE);

// For each 20ms frame: encode -> prepend 2 byte length -> append to output
while (offset + FRAME_BYTES <= pcmBytes.length) {
  const frame = pcmBytes.subarray(offset, offset + FRAME_BYTES);
  const opusData = encoder.encode(frame, FRAME_SAMPLES);
  const lenBuf = new Uint8Array(2);
  new DataView(lenBuf.buffer).setUint16(0, opusData.length, true);
  outputParts.push(lenBuf, opusData);
  offset += FRAME_BYTES;
}
```

Parallel encode to reduce waiting

As a further optimisation, the encoding call is started while Gemini is still generating its response rather than waiting for the full response before encoding. The first chunk of audio is sent to the encoder as soon as it accumulates, running in parallel with the remaining generation. By the time Gemini finishes, most of the audio is already encoded.

VoiceProcessingService.ts | Parallel encode pipeline

```
// VoiceProcessingService.ts | Parallel encode pipeline
// Encoding runs concurrently with Gemini generation

const encodeQueue: Array<Promise<ArrayBuffer>> = [];

// Each time a chunk of Gemini audio arrives, fire an encode immediately
const onChunk = (pcm24: ArrayBuffer) => {
  const pcm16 = GeminiLiveService.downsample24to16(pcm24);
  // ... accumulate into pendingPcm16 ...
  if (pendingPcm16.byteLength >= STREAM_CHUNK_BYTES) {
    // Start HTTP encode now | result awaited in order below
    encodeQueue.push(this.encodePcmToOpus(chunk.buffer));
  }
};

// Wait for Gemini to finish, then send chunks in order as each encodes
await GeminiLiveService.endUserTurn(30_000, onChunk);

for (let i = 0; i < encodeQueue.length; i++) {
  const opusChunk = await encodeQueue[i]; // Already done in most cases
  ESP32WiFiService.sendOpusChunk(new Uint8Array(opusChunk));
}
```

7. Streaming Experiment | What We Attempted and Why It Did Not Move Forward

The most visible latency in the current pipeline is that the ESP32 cannot start playing until the complete Opus file has been received. To address this we designed and implemented a ring buffer streaming approach where the firmware would begin playing audio as soon as the first decoded frames arrived, rather than waiting for the whole file.

The design was architecturally sound and was fully implemented on both the app side and the firmware side. On device testing showed it was not stable on the ESP32-S3 hardware with our current setup. After evaluating the root causes we made the decision to set the streaming approach aside and keep the proven stable pipeline.

How the ring buffer design worked

The firmware would decode each incoming Opus packet immediately upon receipt and write the decoded audio into a ring buffer allocated in PSRAM (the ESP32 external memory). A dedicated FreeRTOS task running on the second CPU core would read from the ring buffer and drive the speaker continuously, starting after a small initial buffer of 60 ms was available.

Arduino firmware | Ring buffer streaming design

```
// Arduino firmware | Ring buffer design overview

// PSRAM ring buffer | 8192 stereo pairs = 512ms of audio capacity
static int16_t*      streamRingBuf      = nullptr;
static volatile uint32_t streamRingWritePos = 0; // Written by WiFi task (Core 0)
static volatile uint32_t streamRingReadPos  = 0; // Read by play task (Core 1)
static volatile bool   streamRingDone      = false;

// Core 0: as each TCP chunk arrives, decode Opus and write to ring
void writeChunkToRingBuf(const uint8_t* data, size_t len) {
    // ... reassemble Opus packets across TCP reads ...
    int decoded = opus_decode(streamDec, opusPkt, pktLen, pcmMono, 960, 0);
    // Write stereo pairs to ring buffer
    for (int s = 0; s < decoded; s++) {
        uint32_t idx = (streamRingWritePos & STREAM_RING_MASK) * 2;
        streamRingBuf[idx]   = pcmMono[s];
        streamRingBuf[idx+1] = pcmMono[s];
        streamRingWritePos++;
    }
    // Start play task once 60ms of audio is buffered (3 frames)
    if (!streamPlayStarted &&
        (streamRingWritePos - streamRingReadPos) >= STREAM_PLAY_THRESHOLD) {
        xTaskCreatePinnedToCore(streamPlayTask, ..., 1);
    }
}
```


Arduino firmware | streamPlayTask()

```
// Arduino firmware | streamPlayTask | Core 1
// Reads ring buffer continuously and writes to I2S speaker

void streamPlayTask(void* param) {
    // Wait for lullaby task to release the I2S speaker hardware
    while (babyTunePlaying) vTaskDelay(pdMS_TO_TICKS(5));

    static int16_t playBuf[512 * 2];

    for (;;) {
        uint32_t filled = streamRingWritePos - streamRingReadPos;
        if (filled == 0) {
            if (streamRingDone) break;
            vTaskDelay(pdMS_TO_TICKS(5));
            continue;
        }
        uint32_t take = (filled > 512) ? 512 : filled;
        for (uint32_t i = 0; i < take; i++) {
            uint32_t idx = (streamRingReadPos & STREAM_RING_MASK) * 2;
            playBuf[i*2] = streamRingBuf[idx];
            playBuf[i*2+1] = streamRingBuf[idx+1];
            streamRingReadPos++;
        }
        i2s_write(I2S_NUM_1, playBuf, take * 4, &written, pdMS_TO_TICKS(200));
    }
    // Notify app that playback is complete
    xQueueSend(cmdQueue, "__PLAYED:response.opus", pdMS_TO_TICKS(100));
    vTaskDelete(NULL);
}
```

8. Root Cause Summary | Why Streaming Was Not Stable

Four independent hardware and firmware issues combined to make the streaming approach unreliable on the ESP32-S3. Each one individually could have been addressed, but together they represented a significant rework of low level hardware interaction that was outside the scope and timeline of this sprint.

Issue	What Happened	Why It Is Hard to Fix
PSRAM bus contention	WiFi DMA writes and I2S DMA reads share the same PSRAM memory bus. Under load they stalled each other, causing gaps in audio output.	Requires either using internal SRAM for the I2S buffer (limited size) or redesigning the buffer architecture to avoid simultaneous PSRAM access.
I2S task handoff conflict	The lullaby task and the streaming play task both used the same I2S hardware. On some runs they accessed it simultaneously, causing a firmware crash.	Requires a strict hardware ownership protocol: the lullaby task must be fully terminated before the play task makes any I2S call.

TCP reassembly edge case	The 2 byte Opus length header occasionally arrived split across two TCP reads. Under load this produced an incorrect expected length and corrupted the decode.	Solvable but requires careful stateful parsing with atomic flag protection across FreeRTOS context switches.
Ring buffer overflow under burst traffic	WiFi delivers data in bursts. The ring buffer filled faster than the speaker could consume it, causing the write pointer to overwrite unplayed audio.	Requires back pressure: the firmware must pause TCP reads when the buffer is near capacity and resume when space is available.

ENGINEERING DECISION

Engineering decision

Given the complexity of the hardware interactions and the stability of the current pipeline, we made the engineering call to revert to the proven store then play approach and deliver a solid, reliable release rather than an unstable fast one.

9. Stable Architecture | What Is Running Now

The current production build uses the LittleFS store then play path. The complete Opus audio file is written to device storage as it arrives. Once the DONE signal is received from the app, the file is decoded frame by frame and played through the speaker. This approach has zero data corruption, is crash free, and handles edge cases reliably.

Arduino firmware | Current stable playback path

```
// Arduino firmware | Stable playback path

// 1. Receive all Opus packets | write each to LittleFS (device storage)
void writeChunkToLittleFS(const uint8_t* data, size_t len) {
  if (ttsRxOpen) ttsRxFile.write(data, len);
}

// 2. On DONE signal from app | full file is on storage | start playback
void exitStreamPlayback(const String& fname) {
  if (babyTunePlaying) stopBabyTune();
  if (ttsRxOpen) { ttsRxFile.close(); ttsRxOpen = false; }
  rxState = RxState::COMMAND;

  // Decode and play frame by frame | zero gaps, zero corruption
  playOpusFromLittleFS(TTS_RX_PATH);

  // Confirm playback complete to app
  tcpSendRaw("PLAYED:" + fname + "\n");
  tcpSendRaw("STATUS:Ready\n");
}
```

10. Sprint Deliverables

Deliverable	Status	Notes
BLE to WiFi TCP transport	Complete	Raw TCP socket port 8765. Faster and more reliable than BLE.
Automatic device discovery	Complete	UDP broadcast, mDNS, and fallback IP. No manual configuration needed.
Native Gemini audio to audio pipeline	Confirmed and running	gemini-2.0-flash-live-001 BidiGenerateContent WebSocket.
Warm WebSocket session	Complete	Session kept open between turns. Connection cost paid once.
Parallel Opus encoding	Complete	Encoding starts during generation. No sequential wait.
Stable Opus decode and playback	Complete	LittleFS store then play. Crash free.
Ring buffer streaming (experiment)	Evaluated not taken forward	Designed and built. Hardware instability on ESP32. Documented.
Release APK	Built and ready	android/app/build/outputs/apk/release/app-release.apk
Automated test suite	Passing	Unit tests for WiFi transport and device discovery.

11. Response Latency | Current Profile and Honest Assessment

LATENCY BASELINE

The current end to end latency is 7 to 8 seconds.

This is the stable, tested production baseline.

It reflects the real time cost of AI generation plus full audio playback.

The 7 to 8 second figure is dominated by two real time processes that cannot be shortened without changing the playback architecture: Gemini model generation time (500 ms to 900 ms first token, plus the time to generate the full spoken response), and speaker playback time (the response must finish generating before it can start playing). For a 5 second spoken response, the pipeline will always take at least 5 seconds of playback on top of the generation latency.

For the use case of a children's AI toy, a conversational pause of 7 to 8 seconds between question and answer is consistent with how children naturally interact with a character waiting to respond. The system is reliable, the audio quality is clear, and the interaction model works.

Scenario	Latency	Notes
Short response (1 to 2 sentences)	5 to 6 seconds	Less generation time, shorter playback
Medium response (3 to 5 sentences)	7 to 8 seconds	Typical interaction. This is the baseline.

Long response (story, explanation)	10 to 15 seconds	Playback time is the dominant factor
------------------------------------	------------------	--------------------------------------

Response length is the biggest variable. Tuning the system prompt to keep responses to 2 to 3 sentences will consistently keep latency at the lower end of the range.

12. Next Steps

The following items are the recommended next actions for the project.

- **APK installation and device testing.** The release APK is ready. Install it on a test device connected to the same WiFi network as the ESP32 and run a full voice interaction cycle. Confirm the 7 to 8 second latency matches expectations and that audio quality is acceptable.
- **System prompt tuning.** The Gemini Live system prompt controls the toy's personality, response length, and topic focus. Shorter responses will reduce the perceived latency significantly. Tuning the prompt to produce 2 to 3 sentence answers will keep typical interactions under 6 seconds.
- **End to end QA across edge cases.** Test WiFi reconnection after sleep, device discovery on different router configurations, and behaviour when the AI response is very short or very long.

This report was prepared by the Machadev development team. All code and firmware referenced in this document is in the project repository. The release APK is at `android/app/build/outputs/apk/release/app-release.apk`.